

*"Either everything happens, or nothing does
-- that is the promise of a transaction."*

MODULE 27

Transaction Management

Ensure reliable, consistent, and secure data operations in enterprise Spring applications with ACID guarantees, @Transactional, and disciplined rollback design.





WHY TRANSACTIONS MATTER (1 OF 2)

Transactions ensure that a set of database operations either all succeed or all fail as a single unit.

This guarantees data integrity, consistency, and reliability -- especially in critical business processes.

TRANSACTIONS ARE ESSENTIAL FOR

**Financial Operations**
Transfers, payments, and reconciliations.

**Order Processing**
Create order, update inventory, charge customer.

**Account Management**
Create/update accounts and related details.

**Reporting Accuracy**
Consistent and reliable data for business decisions.

**Data Integrity & Compliance**
Meet regulatory requirements; protect customer data.

WITHOUT VS. WITH TRANSACTIONS

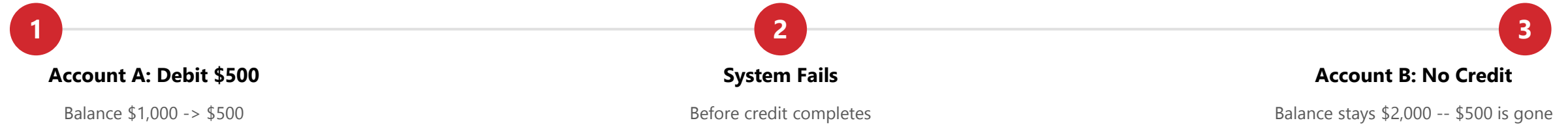
Scenario	Impact
Without Transactions	Partial updates can occur, leading to incorrect balances, data corruption, and loss of trust.
With Transactions	Your data remains accurate, consistent, and trustworthy -- even in the face of failures.

KEY TAKEAWAY Transactions are the foundation of reliable enterprise applications -- protecting your data and your customers.

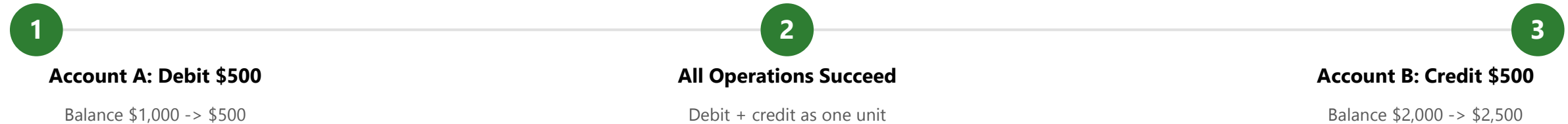
WHY TRANSACTIONS MATTER (2 OF 2)

Real-world impact: the same \$500 transfer, with and without a transaction guarantee.

SCENARIO WITHOUT TRANSACTIONS



SCENARIO WITH TRANSACTIONS



Protects Data Integrity

Ensures Customer Trust

Supports Reliable Business Operations

KEY TAKEAWAY Transactions ensure 'all or nothing' execution -- critical for data integrity and business continuity.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to understand and apply transaction management to ensure data integrity, consistency, and reliability.

- **Understand Transaction Fundamentals** — define a transaction and explain why transactions are critical for data integrity in enterprise systems.
- **Explain ACID Properties** — describe Atomicity, Consistency, Isolation, and Durability and their role in reliable transactions.
- **Explore Spring Transaction Architecture** — understand how Spring manages transactions using PlatformTransactionManager and declarative transaction management.
- **Use @Transactional Effectively** — apply the @Transactional annotation to manage transactions declaratively in Spring applications.
- **Manage Transaction Lifecycle and Behavior** — understand the lifecycle of a transaction, including commit, rollback, and transaction boundaries.
- **Configure Propagation and Isolation** — configure and apply different propagation behaviors and isolation levels based on business requirements.
- **Identify Common Transaction Problems** — recognize issues such as deadlocks, timeouts, and inconsistent reads, and how to mitigate them.
- **Apply Best Practices in Transaction Management** — follow best practices for designing robust, scalable, and maintainable transactional applications.

KEY TAKEAWAY You will be able to design and implement reliable transactions that protect data consistency.

WHAT IS A TRANSACTION?

A transaction is a logical unit of work made up of one or more database operations.

The operations are treated as a single unit that must either all succeed (commit) or all fail (rollback). Transactions prevent partial updates and keep the database in a consistent state.

REAL-WORLD ANALOGY: A BANK TRANSFER

- Transferring money from Account A to Account B involves two operations: (1) debit money from Account A, (2) credit money to Account B.
- **Successful transaction** — both debit and credit happen; money is safely transferred.
- **Failed transaction** — if something goes wrong, neither happens; no money is lost.

A TRANSACTION IN ACTION

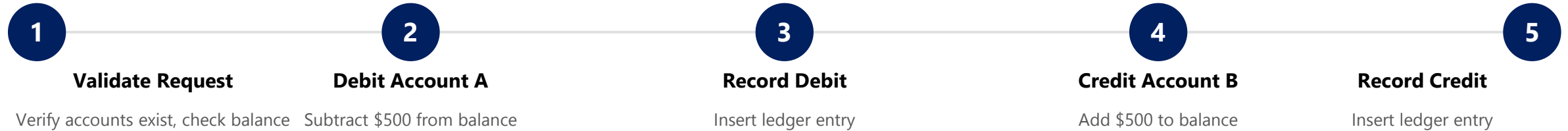
Path	Outcome
Commit (all succeed)	Changes are saved; database updated consistently.
Rollback (any fails)	All changes are undone; database remains unchanged.

KEY TAKEAWAY A transaction guarantees that either all operations complete, or none of them are applied.

REAL-WORLD BANKING EXAMPLE (1 OF 2)

Consider a funds transfer from Account A to Account B for \$500 -- a single business process made of multiple database operations.

FUNDS TRANSFER -- STEP BY STEP



KEY IDEA

Either all five steps are completed successfully, or none of them are applied. This ensures accurate balances and prevents money from appearing or disappearing.

KEY TAKEAWAY These steps are executed as a single transaction to maintain data integrity.

REAL-WORLD BANKING EXAMPLE (2 OF 2)

Success (commit) and failure (rollback) outcomes for the \$500 transfer, and why it matters.

Path	Account A Balance	Account B Balance
Success -- Commit	\$1,000 - \$500 = \$500	\$2,000 + \$500 = \$2,500
Failure -- Rollback	\$1,000 (no change)	\$2,000 (no change)

WHAT HAPPENS ON EACH PATH

- **Success -- Commit** — all operations complete successfully; changes are permanently saved; the database is updated consistently.
- **Failure -- Rollback** — if any step fails (network issue, system error), the entire transaction is rolled back; no changes are saved; data remains consistent.

Protects Data Integrity

Ensures Customer Trust

Supports Reliable Business Operations

KEY TAKEAWAY Transactions protect customer money, ensure accurate reporting, and maintain trust in banking systems.

ACID PROPERTIES OVERVIEW

ACID is a set of four properties that guarantee reliable transaction processing in database systems.

Together, they ensure data remains accurate, consistent, and trustworthy even in the event of failures or concurrent access.

**Atomicity**
All operations succeed, or none are applied. All or nothing.

**Consistency**
The database moves from one valid state to another. Always valid.

**Isolation**
Concurrent transactions do not interfere with each other.

**Durability**
Once committed, results are permanently saved. Permanent changes.

Property	One-Line Example
Atomicity	If a transfer fails after debiting Account A but before crediting Account B, the debit is rolled back.
Consistency	A transaction will not violate rules such as 'account balance cannot be negative.'
Isolation	When two users transfer money at the same time, each sees a consistent view without interference.
Durability	After a successful transfer and commit, even a server crash does not lose the updated balances.

KEY TAKEAWAY ACID properties are the foundation of reliable transactions: correct, consistent, isolated, and permanent.

ATOMICITY

Atomicity ensures that all operations within a transaction are treated as a single, indivisible unit of work.

Either all operations are completed and committed, or none are applied at all. There are no partial updates.

EXAMPLE: \$500 BANK FUNDS TRANSFER



Path	Success (All or Nothing)	Failure (Rollback)
Result	All operations succeed; transaction commits; changes are saved.	Any operation fails; entire transaction rolls back; nothing changes.

KEY TAKEAWAY Atomicity guarantees that a transaction either fully succeeds or leaves the system exactly as it was before it started.



CONSISTENCY

Consistency ensures a transaction brings the database from one valid state to another, obeying all business rules and constraints.

It guarantees that the database never ends up in an invalid state -- every committed transaction leaves the data accurate and rule-compliant.

EXAMPLE: BEFORE AND AFTER A \$500 TRANSFER

Metric	Before Transaction	After Transaction (Valid State)
Account A Balance	\$1,000	\$500
Account B Balance	\$2,000	\$2,500
Total Money in the Bank	\$3,000	\$3,000 -- unchanged

CONSISTENT VS. INCONSISTENT TRANSACTIONS

- **Consistent (valid state)** — all rules are followed; total money in the bank never changes; the database remains valid.
- **Inconsistent (invalid state)** — a rule is violated or a constraint fails (e.g. a negative balance); the change is rejected.

KEY TAKEAWAY Consistency is about correctness -- every committed transaction leaves the database valid and accurate.



ISOLATION

Isolation ensures that concurrent transactions do not interfere with each other -- each appears to run alone.

Intermediate changes made by one transaction are not visible to other transactions until it is committed.

EXAMPLE: TWO USERS TRANSFERRING AT ONCE

- T1 (User A) reads Account balance = \$1,000, prepares to debit \$300, then commits.
- T2 (User B) reads the same account. Isolation ensures T2 cannot see T1's uncommitted change -- T2 reads \$1,000, not \$700, until T1 commits.
- Both transactions run as if they are the only one using the database.

WITHOUT ISOLATION -- PROBLEMS

Problem	What Happens
Dirty Read	T2 reads data T1 wrote but has not committed; if T1 rolls back, T2 read invalid data.
Non-Repeatable Read	T2 reads the same row twice, gets different results because T1 committed changes in between.
Phantom Read	T2 re-runs a query and gets a different set of rows because T1 inserted or deleted rows.

Analogy: think of two accountants working in separate rooms on the same ledger. Neither sees the other's updates until one finalizes and shares the results.

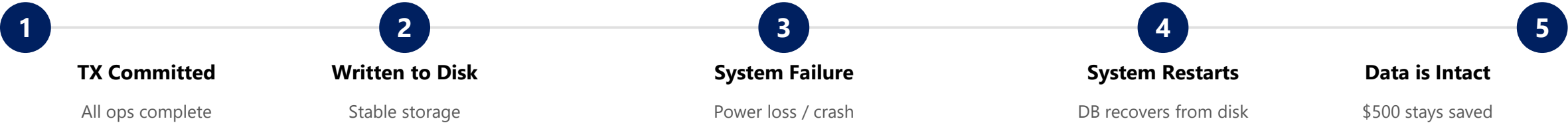
KEY TAKEAWAY Isolation prevents dirty reads, non-repeatable reads, and phantom reads until a transaction commits.

DURABILITY

Durability ensures that once a transaction has been committed, its changes are permanently saved and survive system failures.

Committed data is permanent and cannot be lost -- even in the face of power loss, crashes, or hardware failures.

EXAMPLE: FUNDS TRANSFER WITH SYSTEM FAILURE



Scenario	Before Failure	After Recovery
With Durability	A: \$500 · B: \$2,500 (committed)	A: \$500 · B: \$2,500 -- data persisted on disk
Without Durability	A: \$500 · B: \$2,500 (not written to disk)	A: \$1,000 · B: \$2,000 -- committed changes lost

KEY TAKEAWAY Durability guarantees that once data is committed, it remains safe and persistent no matter what happens next.

TRY IT YOURSELF -- EXERCISE 1: ACID FOR CRM TRANSFERS

Analysis exercise -- tie each ACID letter to a real Northstar CRM transfer observation, not a textbook definition.

Goal: for each ACID letter, write one sentence grounded in real Northstar CRM transfer accounts -- not a restated definition.

STEP-BY-STEP

Step	What To Do
1 -- Fill ACID	In notes/acid-crm.md, write one CRM sentence per ACID letter (Atomicity, Consistency, Isolation, Durability).
2 -- Check the reference	Align: Atomicity = debit+credit+log all succeed or none; Consistency = balances never violate invariants; Isolation = concurrent transfers never see half-updates; Durability = the committed log survives a restart.
3 -- List the accounts	Name all four accounts you'll reference: ACC-1001-MAIN, ACC-1001-LOYALTY, ACC-1002-MAIN, and the synthetic force id ACC-FORCE-FAIL.
4 -- State the boundary	Write one line distinguishing scope: this pre-lab explains ACID in words; Lab 27 proves rollback with real @Transactional code.

PASS CRITERIA

- All four ACID letters explained in CRM-specific language, not generic definitions.
- The synthetic force-fail account (ACC-FORCE-FAIL) is listed alongside the three real accounts.
- The pre-lab vs. Lab 27 boundary is stated clearly: words now, code proof later.

KEY TAKEAWAY TRY IT NOW -- complete Exercise 1 (exercises/exercise-01-acid-crm.md) before continuing: four ACID sentences, four accounts named, one boundary line.

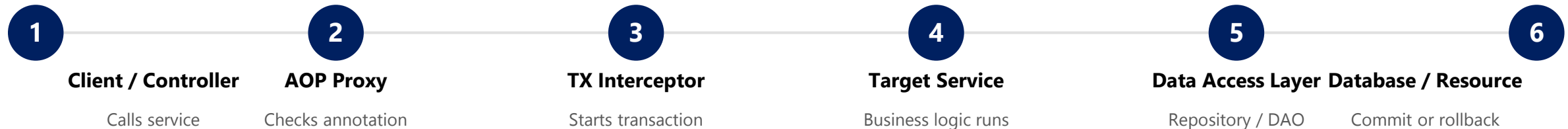
SPRING TRANSACTION ARCHITECTURE (1 OF 2)

Spring provides a declarative transaction management abstraction that simplifies transaction handling across your application layers.

KEY COMPONENTS

- **@Transactional** — declarative annotation defining transaction boundaries.
- **AOP Proxy** — creates a proxy around Spring beans to intercept transactional methods.
- **PlatformTransactionManager** — manages begin, commit, and rollback using the underlying resource.
- **Data Source / Resource** — the actual resource (JDBC, JPA, JMS) where data is persisted.

SPRING TRANSACTION FLOW



Centralized Management

Declarative

Consistent & Reliable

Enterprise Ready

KEY TAKEAWAY Spring uses AOP proxies and PlatformTransactionManager to deliver ACID guarantees with almost no boilerplate.

SPRING TRANSACTION ARCHITECTURE (2 OF 2)

How the flow works step by step, and what happens on the success and exception paths.

HOW IT WORKS, STEP BY STEP

- 1. Client calls a service method.
- 2. The AOP proxy intercepts the call and checks for @Transactional.
- 3. The transaction interceptor starts a transaction before method execution, using PlatformTransactionManager.
- 4. The target service executes its business logic; data access happens via Repository/DAO.
- 5. The data access layer performs database operations using EntityManager or JDBC.
- 6. On success, the transaction is committed; on failure, it is rolled back.
- 7. Control returns to the AOP proxy, and then back to the client.

OUTCOME

Path	What Happens
Success	Transaction committed; all changes saved (Durability); ACID properties are guaranteed.
Exception / Failure	Transaction rolled back; no changes are saved; the system remains consistent.

KEY TAKEAWAY Spring's declarative transaction management works consistently across JDBC, JPA, JMS, and more.

@TRANSACTIONAL ANNOTATION (1 OF 2)

@Transactional is a declarative annotation in Spring that tells Spring to automatically start, commit, or roll back a transaction.

WHERE TO USE

- **At Method Level** — applied to a specific method; only that method runs in a transaction.
- **At Class Level** — applied to the entire class; all public methods run in a transaction unless overridden.

DEFAULT BEHAVIOR

- Starts a transaction before method execution.
- Commits if the method completes successfully.
- Rolls back if a RuntimeException or Error occurs (checked exceptions do NOT roll back by default).
- Uses default propagation = REQUIRED, isolation = DEFAULT, readOnly = false.

Method-Level @Transactional

```
@Service
public class TransferService {

    @Transactional
    public void transfer(String fromId, String toId,
                        BigDecimal amount) {
        accounts.debit(fromId, amount);
        accounts.credit(toId, amount);
    }
}
```

Class-Level @Transactional

```
@Service
@Transactional
public class AccountService {
    public void debit(String id, BigDecimal amt) { /* ... */ }
    public void credit(String id, BigDecimal amt) { /* ... */ }
}
```

KEY TAKEAWAY @Transactional is declarative -- but by default it only rolls back on unchecked exceptions.

@TRANSACTIONAL ANNOTATION (2 OF 2)

Common configurable attributes, the exception/rollback example, and best practices -- including the self-invocation trap.

COMMON ATTRIBUTES

- **propagation** — defines how the transaction behaves with an existing transaction.
- **isolation** — defines the isolation level of the transaction.
- **timeout** — sets the maximum time (seconds) for the transaction to complete.
- **readOnly** — hints that the transaction is read-only; may improve performance.
- **rollbackFor / noRollbackFor** — defines which exceptions should (or should not) trigger rollback.

BEST PRACTICES

- Place @Transactional on service-layer methods.
- Keep transactions short and focused; avoid self-invocation (calling another @Transactional method from within the same class skips the proxy).

Exception & Rollback Example

```
@Transactional
public void withdraw(String id, BigDecimal amount) {
    accounts.debit(id, amount);
    if (amount.compareTo(BigDecimal.ZERO) > 0
        && insufficientFunds(id)) {
        throw new InsufficientFundsException(id);
    }
}
// Result: transaction rolled back.
// No changes are saved to the database.
```

SELF-INVOCATION PITFALL

Calling this.someTxMethod() from inside the same class bypasses the AOP proxy -- @Transactional on that method is silently ignored. Always call through an injected bean.

KEY TAKEAWAY Configure propagation, isolation, timeout, and rollback rules deliberately -- self-invocation skips them all.

TRY IT YOURSELF -- EXERCISE 2: TRANSACTION BOUNDARY PLACEMENT

Architecture exercise -- decide exactly where the transfer transaction boundary belongs, and defend the choice.

Goal: choose the correct home for the transfer's transaction boundary -- service, controller, or repository -- and order the steps inside it.

STEP-BY-STEP

Step	What To Do
1 -- Choose	In notes/tx-boundary.md, pick the boundary for debit+credit+log: TransferService.transfer(...), the controller method, or the repository.
2 -- Check the reference	Confirm: TransferService.transfer(...) is preferred; a controller method should be avoided; repository-only is too narrow for a multi-step business operation.
3 -- Order the internal steps	Write the order inside the boundary: load accounts -> debit -> credit -> write TransactionLog.
4 -- Note the correlation	Record that the happy-path evidence uses correlation id lab-request-001.

WHERE @TRANSACTIONAL BELONGS

TransferService.java vs. TransferController.java

```
// Correct -- the service owns the transaction boundary
@Service
public class TransferService {
    @Transactional
    public TransactionLog transfer(String fromId, String toId, BigDecimal amount) { /* debit, credit, log */ }
}

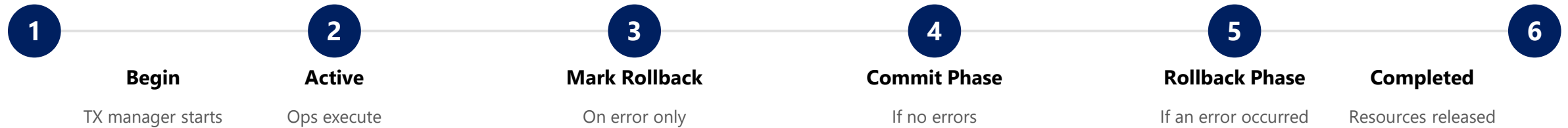
// Wrong -- controllers must stay thin; @Transactional does NOT belong here
@RestController public class TransferController { /* delegates to TransferService */ }
```

KEY TAKEAWAY TRY IT NOW -- complete Exercise 2 (exercises/exercise-02-transaction-boundary.md): service chosen over controller, four steps ordered, correlation id noted.

TRANSACTION LIFECYCLE (1 OF 2)

A transaction goes through a well-defined lifecycle from start to completion, handled automatically by Spring and the transaction manager.

THE TRANSACTION LIFECYCLE



KEY POINTS

- Transactions ensure data integrity by grouping multiple operations as a single unit of work.
- Either all operations succeed (COMMIT) or all are undone (ROLLBACK).
- The transaction manager controls the lifecycle automatically.
- You focus on business logic -- Spring handles the transaction details.

KEY TAKEAWAY Understanding the transaction lifecycle helps you write more reliable, data-safe applications.

TRANSACTION LIFECYCLE (2 OF 2)

The success path versus the error path, and what happens behind the scenes on every transaction.

SUCCESS PATH -- COMMIT



ERROR PATH -- ROLLBACK



BEHIND THE SCENES



KEY TAKEAWAY Whether commit or rollback, the lifecycle always ends the same way: resources released and control returned to the caller.

COMMIT OPERATIONS

A commit operation permanently saves all changes made in a transaction when it completes successfully.



TransferService.java

```
@Transactional
public TransactionLog transfer(String fromId, String toId,
                               BigDecimal amount) {
    accounts.debit(fromId, amount);
    accounts.credit(toId, amount);
    // Spring commits automatically if no exception is thrown
    return logs.save(new TransactionLog(...));
}
```

- Makes data changes permanent.
- Ensures data consistency.
- Maintains ACID properties.
- Releases database locks.
- Allows other transactions to see the committed data.

KEY TAKEAWAY Commit makes all changes permanent and visible to other transactions -- and releases locks so other work can proceed.

ROLLBACK OPERATIONS

A rollback operation undoes all changes made during a transaction when an error occurs.



TransferService.java (Rollback Trigger)

```
@Transactional
public TransactionLog transfer(String fromId, String toId,
                              BigDecimal amount) {
    accounts.debit(fromId, amount);
    accounts.credit(toId, amount);
    if ("ACC-FORCE-FAIL".equals(toId)) {
        throw new IllegalStateException("Forced failure");
    }
    // Any RuntimeException here rolls back debit AND credit.
}
```

- An unhandled exception is thrown during execution.
- Validation fails (e.g. insufficient funds).
- A database error occurs (e.g. deadlock).
- Programmatic rollback via `setRollbackOnly()`.
- A timeout or system-level failure occurs.

KEY TAKEAWAY Rollback ensures partial changes are never saved -- the database returns to a consistent, pre-transaction state.

TRY IT YOURSELF -- EXERCISE 3: ROLLBACK EVIDENCE PLAN

Documentation exercise -- plan exactly what must stay unchanged after a forced ACC-FORCE-FAIL failure.

Goal: document the happy path and the forced-failure path side by side, plus one rule for rejecting unsafe AI-drafted exception handling.

STEP-BY-STEP

Step	What To Do
1 -- Happy path	In notes/rollback-plan.md, describe: MAIN -> LOYALTY updates both balances and writes one TransactionLog row.
2 -- Force fail	Describe: a transfer to ACC-FORCE-FAIL leaves the MAIN balance unchanged and writes no success log row.
3 -- Test idea	Sketch (plan only, no code yet) an automated test that asserts account balances after the forced failure.
4 -- AI caution	Write the rejection rule: reject any AI draft that catches Exception and swallows it inside a @Transactional method.

WHY SWALLOWED EXCEPTIONS BREAK ROLLBACK

Reject This AI-Drafted Pattern

```
@Transactional
public void transfer(String fromId, String toId, BigDecimal amount) {
    try {
        accounts.debit(fromId, amount);
        accounts.credit(toId, amount);
    } catch (Exception e) {
        log.warn("transfer issue", e); // swallowed -- method returns normally
    }
    // Spring sees no exception propagate out -> COMMITS the partial debit. Atomicity broken.
```

KEY TAKEAWAY TRY IT NOW -- complete Exercise 3 (exercises/exercise-03-rollback-plan.md): happy path and fail path contrasted, no-log-on-fail stated, swallowed-exception rule written.

TRY IT YOURSELF -- EXERCISE 4: TRANSFER PSEUDOCODE (STARTER)

Hands-on exercise -- fill in the TODOs in a starter TransferService sketch; this is a STARTER file, not a finished solution.

Goal: complete a TODO-riddled pseudocode sketch that debits, credits, and rolls back on a forced ACC-FORCE-FAIL failure.

STEP-BY-STEP

Step	What To Do
1 -- Create the file	Create notes/TransferServiceSketch.java in your exercises workspace.
2 -- Fill the TODOs	Replace every blank: @Transactional for one unit of work, IllegalStateException (or RuntimeException) for the forced failure, and amount for both debit/credit calls.
3 -- Self-check	Write one sentence explaining why a caught-and-ignored exception would break atomicity here.
4 -- Reflect	Note that CUS-1001 / CUS-1002 own the seeded accounts -- do not invent a Kafka outbox for this exercise.

FILL THE BLANKS

TransferServiceSketch.java (STARTER -- not a solution)

```
// Sketch only -- not a full Spring project
class TransferService {
    // TODO: Spring annotation for one unit of work
    @_____
    void transfer(String fromId, String toId, long amount) {
        Account from = load(fromId);
        Account to = load(toId);
        if ("ACC-FORCE-FAIL".equals(toId)) {
            // TODO: throw a runtime exception to trigger rollback
            throw new _____("forced failure");
        }
        from.debit(_____); // TODO: amount
        to.credit(_____); // TODO: amount
        logSuccess(fromId, toId, amount);
    }
}
```

KEY TAKEAWAY TRY IT NOW -- complete Exercise 4 (exercises/exercise-04-transfer-pseudocode.md) -- a STARTER file, not a solution: every blank filled and the atomicity reflection written.

PROPAGATION TYPES (1 OF 2)

Propagation defines how a transactional method behaves when called from within another transactional method.

It determines whether to join an existing transaction, create a new one, suspend the current one, or throw an exception. REQUIRED is Spring's default.

Type	Behavior	When Called From Another Transaction
REQUIRED (default)	Joins the existing transaction if one exists; otherwise creates a new one.	Runs within the same transaction as the caller.
REQUIRES_NEW	Suspends the existing transaction (if any); creates a new, independent transaction.	Runs in its own transaction, independent of the caller -- caller's transaction is suspended, then resumed.
SUPPORTS	Joins the existing transaction if one exists; otherwise runs without a transaction.	Executes non-transactionally if no transaction exists.

KEY POINTS

- Propagation controls the scope and boundaries of a transaction -- essential when one @Transactional method calls another.
- REQUIRED is Spring's default and by far the most commonly used propagation type.

KEY TAKEAWAY REQUIRED joins or creates a transaction and is the right default for almost every service method.

PROPAGATION TYPES (2 OF 2)

Four more propagation types -- MANDATORY, NOT_SUPPORTED, NEVER, and NESTED -- plus practical guidance on choosing between them.

Type	Behavior	When Called From Another Transaction
MANDATORY	Must run within an existing transaction; throws an exception if none exists.	Joins the existing transaction; fails with an exception if there is no caller transaction.
NOT_SUPPORTED	Suspends the existing transaction (if any) and runs non-transactionally.	Always executes without a transaction, even if the caller has one.
NEVER	Must not run within a transaction; throws an exception if one exists.	Fails with an exception if there is an existing caller transaction.
NESTED	Runs in a nested transaction within the current one, using savepoints.	Rolls back to the savepoint without affecting the outer transaction.

CHOOSING THE RIGHT PROPAGATION TYPE

- Use **REQUIRES_NEW** — for audit logging, notifications, or independent operations that must persist regardless of the caller's outcome.
- Use **NESTED** — when working with JDBC savepoints for partial rollback within a larger transaction.
- Use **MANDATORY** — to enforce that a method is only ever called from within an already-open transaction.

KEY TAKEAWAY Choose propagation deliberately -- joining, suspending, nesting, and rejecting all trade off differently.

TRY IT YOURSELF -- EXERCISE 5: PROPAGATION WARNINGS

Analysis exercise -- flag common AOP and propagation mistakes before Lab 27, including the self-invocation trap.

Goal: list three realistic propagation risks for the transfer flow, then state Lab 27's safe default.

STEP-BY-STEP

Step	What To Do
1 -- List the risks	In notes/propagation-warnings.md, flag three risks: NOT_SUPPORTED used mid-transfer, REQUIRES_NEW used for the log only, and self-invocation bypassing the proxy.
2 -- State the default	Confirm the default propagation REQUIRED on the outer transfer() method is enough for this lab -- no custom propagation needed.
3 -- Note the proxy trap	Write why calling this.transfer(...) from inside the same class may skip the Spring AOP proxy entirely.
4 -- Respect the boundary	State that Lab 27 does not require configuring custom transaction managers -- Spring Boot's defaults suffice.

THE SELF-INVOCATION TRAP

AccountFacade.java (bypasses the proxy)

```
@Service
public class AccountFacade {
    @Transactional
    public void transfer(String fromId, String toId, BigDecimal amount) { /* debit, credit, log */ }

    public void batchTransfer(List<TransferRequest> reqs) {
        for (TransferRequest r : reqs) { this.transfer(r.from(), r.to(), r.amount()); } // BUG: skips proxy, @Transactional ignored
    }
}
```

KEY TAKEAWAY TRY IT NOW -- complete Exercise 5 (exercises/exercise-05-propagation-warnings.md): three risks listed, REQUIRED default stated, self-invocation warning written.

ISOLATION LEVELS (1 OF 2)

Isolation level defines how visible uncommitted changes are between concurrent transactions.

Higher isolation levels provide stronger consistency but may reduce concurrency and performance.

Isolation Level	Description	Prevents	Performance
READ_UNCOMMITTED	Allows reading uncommitted data from other transactions.	Nothing	Highest (least restrictive)
READ_COMMITTED	Allows reading only committed data; prevents dirty reads.	Dirty reads	High
REPEATABLE_READ	Rows read remain the same for the duration of the transaction.	Dirty + non-repeatable reads	Medium
SERIALIZABLE	Transactions appear to run serially; strictest isolation.	Dirty + non-repeatable + phantom reads	Lowest (most restrictive)
SNAPSHOT (MVCC)	Uses row versioning for a consistent snapshot without locking reads.	All three, optimistically	High (optimistic)

KEY POINTS

- Prevents concurrency problems like dirty reads, non-repeatable reads, and phantom reads.
- Default isolation in most databases (Oracle, SQL Server, PostgreSQL) is READ_COMMITTED.

KEY TAKEAWAY Higher isolation means stronger consistency but lower concurrency -- pick the lowest level you need.

ISOLATION LEVELS (2 OF 2)

Concurrency anomalies isolation levels defend against, and the real-world defaults across common databases.

CONCURRENCY ANOMALIES



Dirty Read

Reading data written by another transaction that has not yet been committed. Prevented by READ_COMMITTED and above.



Non-Repeatable Read

Reading the same row twice and getting different results because another transaction modified and committed the data. Prevented by REPEATABLE_READ and above.



Phantom Read

Re-running a query and getting additional rows inserted by another committed transaction. Prevented by SERIALIZABLE and SNAPSHOT.

DEFAULTS ACROSS COMMON DATABASES

Database	Default Isolation	Notes
Oracle Database	READ_COMMITTED	No REPEATABLE_READ option; use SERIALIZABLE if needed.
SQL Server	READ_COMMITTED	Also supports SNAPSHOT isolation via row versioning.
MySQL (InnoDB)	REPEATABLE_READ	A common surprise for developers coming from Oracle/SQL Server.
PostgreSQL	READ_COMMITTED	Also supports SERIALIZABLE with true serializable snapshot isolation.

KEY TAKEAWAY Know your database's default isolation level -- MySQL InnoDB's REPEATABLE_READ default surprises many.

COMMON TRANSACTION PROBLEMS (1 OF 2)

Even with ACID properties, concurrent transactions can cause issues that impact data consistency and system performance.

Problem	Cause	Example	Fix
Dirty Read	Occurs at low isolation levels (e.g. READ_UNCOMMITTED).	T1 transfers \$100 but rolls back; T2 already read the uncommitted balance and acted on it.	Use READ_COMMITTED or higher; never expose uncommitted data.
Non-Repeatable Read	Another transaction updates a row and commits between two reads.	T1 reads balance \$500; T2 updates it to \$600 and commits; T1 reads again and gets \$600.	Use REPEATABLE_READ or SERIALIZABLE; lock rows when reading if needed.
Phantom Read	Another transaction inserts or deletes rows matching the query between reads.	T1 queries accounts with balance > \$1,000 (5 rows); T2 inserts a new matching row; T1 re-runs the query and gets 6 rows.	Use SERIALIZABLE isolation; use range locks (index locking).

KEY POINTS

- Concurrency issues occur when multiple transactions access the same data at the same time.
- They can lead to inconsistent or incorrect data if not handled properly -- isolation levels and locking help prevent them.

KEY TAKEAWAY Even with ACID guarantees, concurrent transactions can still create real data-visibility problems.

COMMON TRANSACTION PROBLEMS (2 OF 2)

Lost updates, deadlocks, and long-running transactions -- and the best practices that prevent all five problems.

Problem	Cause	Example	Fix
Lost Update	No locking or version control on the data during update.	T1 and T2 both read balance \$500; both update and commit; T2's commit silently overwrites T1's -- final value is T2's, T1's update is lost.	Use locking (SELECT ... FOR UPDATE) or optimistic locking with a version field.
Deadlock	Improper lock order or cyclic dependencies between transactions.	T1 locks Row A and waits for Row B; T2 locks Row B and waits for Row A -- both wait forever.	Ensure consistent lock ordering; keep transactions short; use deadlock timeout and retry.
Long-Running Transaction	Complex queries, network delays, or external calls inside a transaction.	A reporting transaction runs for several minutes and blocks all updates on the affected tables.	Keep transactions short; move long operations outside the transaction; use proper indexing.

BEST PRACTICES TO PREVENT TRANSACTION PROBLEMS

Choose the Right Isolation Level

Keep Transactions Short

Use Proper Indexes

Access Data in a Consistent Order

Monitor Long-Running Transactions

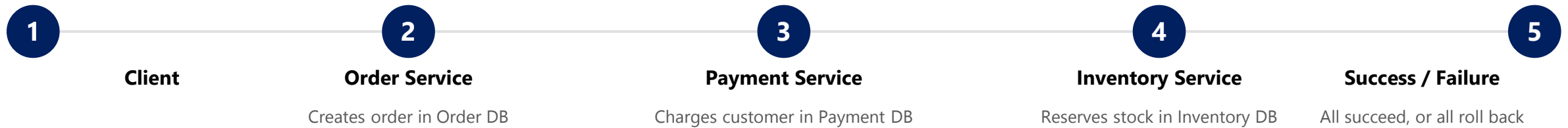
Handle Deadlocks and Retries

KEY TAKEAWAY Lost updates and deadlocks are concurrency risks, not ACID failures -- locking strategy is your defense.

DISTRIBUTED TRANSACTIONS OVERVIEW (1 OF 2)

Distributed transactions span multiple independent resources or services across a network -- a harder, more advanced problem than a single-database transaction.

They ensure that either all participating resources commit or all roll back, maintaining data consistency across the system.



WHY DISTRIBUTED TRANSACTIONS MATTER

- **Data Consistency** — keeps data consistent across multiple independent systems.
- **Business Integrity** — ensures business rules are maintained end-to-end.
- **Complex Workflows** — supports multi-step operations spanning multiple services.
- **Heterogeneous Systems** — works across different databases, platforms, and technologies.

COMMON USE CASES

E-Commerce

Banking

Travel Booking

Insurance

Microservices

KEY TAKEAWAY Distributed transactions guarantee all-or-nothing behavior across services -- but at a much higher cost.

DISTRIBUTED TRANSACTIONS OVERVIEW (2 OF 2)

Two-Phase Commit (2PC), its real challenges, and the alternative Saga pattern.

TWO-PHASE COMMIT (2PC)

- **Phase 1 -- Prepare (Vote):** the coordinator asks all participants to prepare to commit; each responds YES (ready) or NO.
- **Phase 2 -- Commit / Rollback:** if all participants respond YES, the coordinator tells everyone to commit; if any responds NO or times out, everyone rolls back.

CHALLENGES

- **Performance** — extra coordination round-trips increase latency.
- **Single Point of Failure** — coordinator failure can block the transaction.
- **Network Issues** — failures, partitions, or timeouts create uncertainty.
- **Resource Locking** — longer lock duration reduces concurrency.
- **Complexity** — harder to implement, test, and monitor.



Two-Phase Commit (2PC)

Strong consistency using a central coordinator; participants block until the decision arrives.



Saga Pattern

Event-driven approach: a chain of local transactions with compensating actions if a later step fails.

KEY TAKEAWAY Distributed transactions are advanced -- reach for 2PC or Saga once an operation spans independent services.

TRANSACTION BEST PRACTICES (1 OF 2)

Following these best practices builds reliable, consistent, high-performance applications while minimizing failures and contention.

**Consistency First**
Ensure data consistency and integrity across all operations.

**Keep It Short**
Reduce locks and improve throughput.

**Fail Fast**
Validate early to avoid unnecessary rollbacks.

**Handle Exceptions**
Handle exceptions properly; roll back cleanly.

**Design for Scale**
Design with scalability and concurrency in mind.

#	Practice	What It Means
1	Define Clear Boundaries	Include only the operations that must succeed or fail together; avoid overly large transactions.
2	Keep Transactions Short	Minimize the time a transaction holds locks and resources; move non-essential processing outside.
3	Choose Appropriate Isolation Level	Use the lowest isolation level that meets your consistency requirements, to improve concurrency.
4	Validate Early	Validate inputs and business rules before starting the transaction to reduce unnecessary rollbacks.
5	Handle Exceptions Properly	Catch and handle exceptions; ensure rollback happens automatically or programmatically when required.

KEY TAKEAWAY Good transaction design ensures data integrity, performance, and system reliability.

TRANSACTION BEST PRACTICES (2 OF 2)

Practices 6-10, and the common pitfalls that violate them in real codebases.

#	Practice	What It Means
6	Use Appropriate Propagation	Choose the right propagation type (REQUIRED, REQUIRES_NEW, etc.) based on business needs.
7	Avoid Long-Held Locks	Avoid user interaction, external calls, or slow operations inside a transaction.
8	Monitor and Tune	Monitor transaction performance, lock waits, and rollbacks; tune isolation levels, indexes, and queries.
9	Test for Concurrency	Test with concurrent users and loads to identify contention, deep rollbacks, and deadlocks early.
10	Log and Audit	Log key transaction events and errors for troubleshooting and audit compliance.

COMMON PITFALLS TO AVOID

- **Long Transactions** — holding locks too long impacts performance and increases deadlock risk.
- **External Calls Inside a Transaction** — API calls, messaging, or file I/O inside a transaction can cause timeouts and partial failures.
- **Wrong Isolation Level** — too high causes blocking; too low causes inconsistent reads.
- **Ignoring Exceptions** — swallowing exceptions prevents proper rollback and leads to inconsistent data.
- **Incorrect Propagation** — using the wrong propagation type can cause unexpected commits or rollbacks.
- **No Monitoring** — not tracking slow transactions, lock waits, or failures leads to unseen issues.

KEY TAKEAWAY Well-designed transactions ensure data integrity and system reliability -- apply these ten practices consistently.

TRY IT YOURSELF -- EXERCISE 6: LAB 27 READINESS CHECKLIST

Capstone documentation exercise -- confirm dependencies, workspace path, and evidence folders before starting the graded lab.

Goal: ready the lab27-crm project by confirming what Labs 25-26 already gave you, and what stays deferred to Lab 28.

STEP-BY-STEP

Step	What To Do
1 -- Depends on	In notes/lab27-readiness.md, name what Labs 25 and 26 already established: the layered Service/Repository structure and the dev/H2 configuration profile.
2 -- Path	Write the exact workspace path for the graded project: examples/lab27-crm.
3 -- Evidence folders	Plan a screenshots folder for happy-path and rollback evidence: notes/screenshots/lab-27/.
4 -- Defer	State explicitly what is out of scope: JWT protection of the transfer routes waits for Lab 28.

SCOPE BOUNDARY -- DO NOT BUILD LATER TECHNOLOGY YET

- **Do now** — ACID in CRM language, @Transactional on the service (not the controller), debit+credit+log as one unit of work, ACC-FORCE-FAIL rollback evidence.
- **Do not add yet** — JWT SecurityFilterChain (Lab 28), distributed sagas / Kafka transactions (Week 4), or manual EntityManager commit APIs as the primary style.

KEY TAKEAWAY TRY IT NOW -- complete Exercise 6 (exercises/exercise-06-lab27-readiness.md) right before Lab 27: dependencies named, path written, Lab 28 deferred for JWT.

LAB OVERVIEW -- TRANSACTION MANAGEMENT

Lab 27: Transaction Management with AI Assistance -- Northstar CRM Transfers

BUSINESS SCENARIO

- Agents move funds between CRM sub-accounts (e.g. MAIN -> LOYALTY). A debit without a matching credit is an incident.
- Leadership freezes: all multi-account money movement goes through @Transactional TransferService. Controllers must not own transaction boundaries.
- Demonstrate rollback with a synthetic ACC-FORCE-FAIL destination; document ACID with real before/after balances, not slogans.

LEARNING OBJECTIVES

- Place @Transactional on service methods spanning multiple account updates.
- Explain Spring's proxy-based demarcation and why self-invocation skips it.
- Implement a transfer (debit + credit + log) for CRM accounts.
- Force a mid-operation failure and prove both sides roll back.
- Map Atomicity, Consistency, Isolation, and Durability to observable behavior.

CRM FIXTURES USED THROUGHOUT THE LAB

ID	Owner / Role	Seed Balance	Purpose
ACC-1001-MAIN	CUS-1001 Amina Khan (ACTIVE)	\$1,000.00	Happy-path transfer source
ACC-1001-LOYALTY	CUS-1001 Amina Khan	\$100.00	Happy-path transfer destination
ACC-1002-MAIN	CUS-1002 Ravi Singh (PROSPECT)	\$250.00	Secondary account
ACC-FORCE-FAIL	Synthetic sink	n/a	Triggers the rollback demo

KEY TAKEAWAY Correlation id lab-request-001 anchors every transfer example; ACC-FORCE-FAIL proves rollback, not just the happy path.

LAB TASKS AND DELIVERABLES

Northstar CRM Transfers -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Branch prior CRM to lab27-crm; add spring-boot-starter-data-jpa + H2 persistence.
2	Seed Amina/Ravi accounts (MAIN, LOYALTY) with fixed starting balances.
3	Build AccountRepository, TransactionLogRepository, and the TransactionLog entity.
4	Implement @Transactional TransferService (debit + credit + log, one unit of work).
5	Expose POST /api/transfers; prove the happy path MAIN -> LOYALTY.
6	Demonstrate rollback: transfer to ACC-FORCE-FAIL; prove balances are unchanged.
7	Document ACID with real lab evidence (curls, balances, H2 durability caveat).
8	Automated tests + AI review notes (lab27-001); run mvn test twice.

EXPECTED DELIVERABLES

- @Transactional TransferService + thin controller.
- Seeded accounts + TransactionLog entity.
- Happy-path evidence (MAIN->LOYALTY + correlation).
- ACC-FORCE-FAIL rollback evidence (balances + no log).
- ACID explanation tied to real observations.
- Automated tests proving rollback balances.
- AI review notes or manual equivalent; README runbook.

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/AI Review 10 · Docs 10

KEY TAKEAWAY A decreased balance despite a thrown exception, or @Transactional on the controller, both lose real marks.

MODULE 27 SUMMARY

Transaction management ensures that data remains consistent and reliable even in complex, multi-step operations across system boundaries.

WHAT WE COVERED



Transaction Basics & ACID

A logical unit of work, and the four properties that make it reliable.



@Transactional & Spring Architecture

Declarative management via AOP proxies and PlatformTransactionManager.



Commit, Rollback & Lifecycle

How a transaction begins, executes, and always ends one of two ways.



Propagation & Isolation

Controlling transaction boundaries and concurrent visibility.



Common Problems & Distributed TX

Dirty reads, deadlocks, lost updates -- and the harder multi-service case.



Best Practices

Short, well-bounded transactions with the right isolation and clean rollback.

MODULE FLOW RECAP

1

ACID Properties

2

@Transactional

3

Propagation/Isolation

4

Best Practices

5

Lab: CRM Transfers

KEY TAKEAWAY Correctly designed transactions protect data consistency and support reliable, enterprise-ready systems.

KNOWLEDGE CHECK (1 OF 3)

Test your understanding of ACID properties, commit, and rollback.

1. Which property ensures that all steps of a transaction are completed successfully or none are?

- A. Consistency
- B. Atomicity**
- C. Isolation
- D. Durability

3. If a transaction fails, which operation undoes all changes made by that transaction?

- A. Commit
- B. Revert
- C. Rollback**
- D. Reset

2. Which operation makes the changes of a transaction permanent?

- A. Rollback
- B. Savepoint
- C. Commit**
- D. Flush

4. Which propagation type joins an existing transaction if one is already in progress?

- A. REQUIRED**
- B. REQUIRES_NEW
- C. SUPPORTS
- D. NEVER

KEY TAKEAWAY Atomicity is all-or-nothing; COMMIT saves changes permanently; ROLLBACK undoes them entirely.

KNOWLEDGE CHECK (2 OF 3)

Four more questions on isolation levels, propagation, and common concurrency problems.

5. Which isolation level allows dirty reads?

- A. READ_COMMITTED
- B. REPEATABLE_READ
- C. SERIALIZABLE
- D. READ_UNCOMMITTED**

7. Which isolation level prevents non-repeatable reads?

- A. READ_COMMITTED
- B. REPEATABLE_READ**
- C. SERIALIZABLE
- D. READ_UNCOMMITTED

6. Which propagation type always starts a new transaction, suspending any existing transaction?

- A. REQUIRED
- B. REQUIRES_NEW**
- C. MANDATORY
- D. SUPPORTS

8. What is a common problem that can occur with concurrent transactions?

- A. Deadlock**
- B. Cache Miss
- C. Memory Leak
- D. Null Pointer Exception

KEY TAKEAWAY READ_UNCOMMITTED is least restrictive; REQUIRES_NEW always starts independently; deadlocks are a classic risk.

KNOWLEDGE CHECK (3 OF 3)

Two final questions, plus a bonus challenge question on distributed transactions.

9. Which ACID property ensures that once a transaction is committed, it cannot be changed?

- A. Atomicity
- B. Consistency
- C. Isolation
- D. Durability**

10. Which of the following is considered a best practice for transaction management?

- A. Keep transactions long to avoid frequent commits
- B. Swallow exceptions and continue
- C. Handle exceptions and roll back when needed**
- D. Use READ_UNCOMMITTED for better performance always

BONUS CHALLENGE QUESTION (FROM THE CURRICULUM)

"In a distributed system, why is transaction management more complex compared to a single-database system?" -- write your own answer before revealing the model answer below.

- **No single transaction manager** — each service/database has its own local transaction with no shared transaction log.
- **Partial failure across services** — one participant can fail after another has already committed locally.
- **Coordination cost** — patterns like Two-Phase Commit add latency, locking, and a coordinator single point of failure.

KEY TAKEAWAY Durability makes commits permanent; handling exceptions and rolling back correctly is the real best practice.

*"Either everything happens, or nothing does
-- that is the promise of a transaction."*

MODULE 27 COMPLETE

Transaction Management

You can now design and implement reliable @Transactional boundaries -- with ACID guarantees, sensible propagation and isolation, and clean commit/rollback behavior -- for enterprise Spring applications.